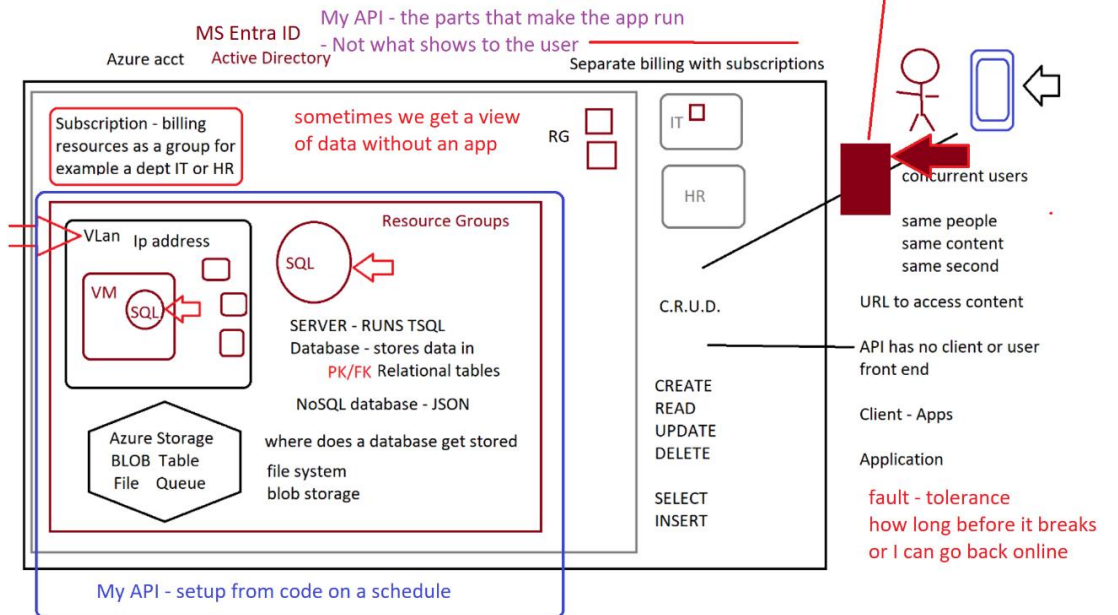


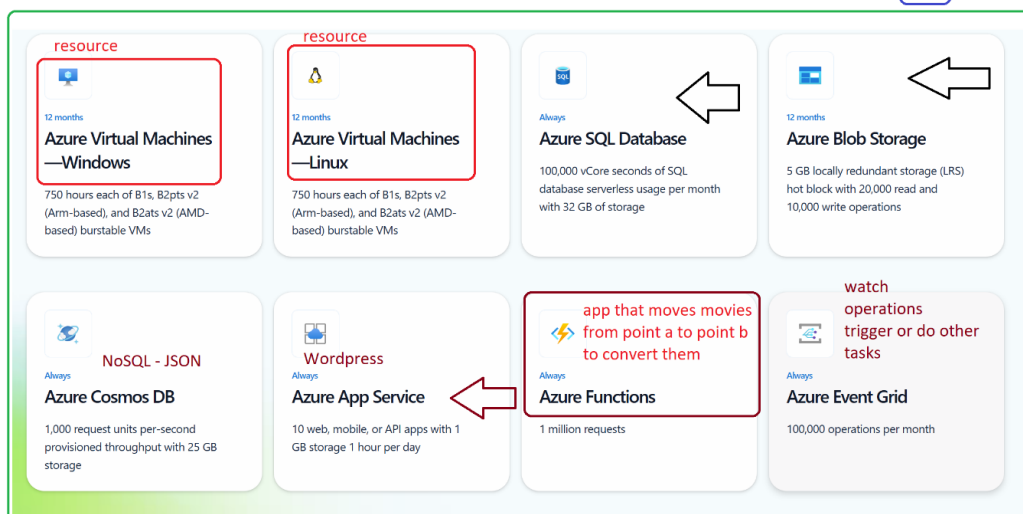
PBI Extras 202X

Parent - Child Relation - Hierarchy or Who Owns What - Tim has many orders
 - One to One - One to Many - Many to Many - Way of Grouping items together

relay race we hand a baton to the next person or step or task



My API → My Resource Group → our app and it does a lot of thing connecting to many

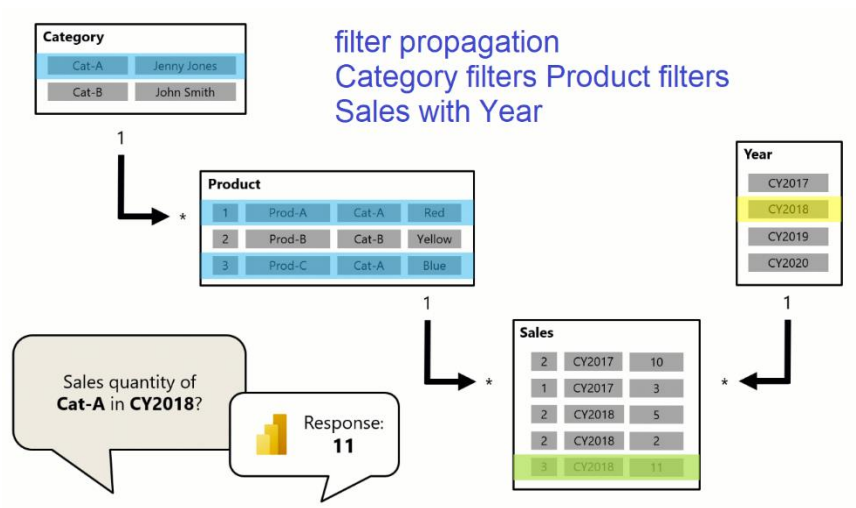


Contents

Discussion.....	3
Notable DAX.....	6
Filter Context	7
USERRELATIONSHIP	7
Measures.....	9
When to use USERRELATIONSHIP	11
TL;DR stands for “Too Long; Didn’t Read.”	12
Toggle Slicer	16
Why this works.....	18
1 YTD/MTD/LY toggles.....	18
Quick steps (recap)	21
Parameters.....	22
Quick chooser: which “parameter” should I use?	22
1) Create the parameter	26
Power BI Forecasting	30
Shared Semantic Model Datasets.....	34
What is unique to the Service/Fabric	35
Dashboards/Workspaces	36
RLS Row Level Security – Report vs Fabric/Service.....	37
Object Level Security – RLS Advanced	37
Azure Functions	38
Azure Dataverse.....	38
Reference.....	45
Templates.....	45
Analyzing & Large Report Example	45

Discussion

1. [DAX sample model - DAX | Microsoft Learn](#)
2. *Examples in this article can be used with the sample Adventure Works DW 2020 Power BI Desktop model. To get the model, see [DAX sample model](#)*
3. [Model relationships in Power BI Desktop - Power BI | Microsoft Learn](#)
4. [CALCULATE function \(DAX\) - DAX | Microsoft Learn](#)
5. [CALCULATETABLE function \(DAX\) - DAX | Microsoft Learn](#)
6. [USERELATIONSHIP function \(DAX\) - DAX | Microsoft Learn](#)
 - a. Cannot be used when row level security is defined for the table in which the measure is included.
 - b. Example: Override the default, active, relationship between InternetSales and DateTime tables.
 - c. To calculate the sum of internet sales and allow slicing by ShippingDate instead of the traditional OrderDate,
 - d. Create measure, [InternetSales by ShippingDate]
 - e. = CALCULATE(SUM(InternetSales[SalesAmount]),
USERELATIONSHIP(InternetSales[ShippingDate], DateTime[Date]))
7. [Tutorial: From Dimensional Model to Stunning Report in Power BI Desktop - Power BI | Microsoft Learn](#)



DAX

Copy

```
CALCULATE(<expression>[, <filter1> [, <filter2> [, ...]])
```

Term	Definition
expression	The expression to be evaluated.
filter1, filter2,...	(Optional) Boolean expressions or table expressions that defines filters, or filter modifier functions.

The expression used as the first parameter is essentially the same as a measure.

Filters can be:

- Boolean filter expressions
- Table filter expressions
- Filter modification functions

Filter modifier functions

Filter modifier functions allow you to do more than simply add filters. They provide you with additional control when modifying filter context.

Function	Purpose
REMOVEFILTERS	Remove all filters, or filters from one or more columns of a table, or from all columns of a single table.
ALL ¹ , ALLEXCEPT, ALLNOBLANKROW	Remove filters from one or more columns, or from all columns of a single table.
KEEPFILTERS	Add filter without removing existing filters on the same columns.
USERELATIONSHIP 	Engage an inactive relationship between related columns, in which case the active relationship will automatically become inactive.
CROSSFILTER	Modify filter direction (from both to single, or from single to both) or disable a relationship.

DAX
Copy

```

Blue Revenue =
CALCULATE(
SUM(Sales[Sales Amount]),
'Product'[Color] = "Blue"
)

```

Category	Sales Amount	Blue Revenue
Accessories	\$1,272,057.89	\$165,406.62
Bikes	\$94,620,526.21	\$8,374,313.88
Clothing	\$2,117,613.45	\$259,488.37
Components	\$11,799,076.66	\$803,642.10
Total	\$109,809,274.20	\$9,602,850.97

The CALCULATE function evaluates the sum of the **Sales** table **Sales Amount** column in a modified filter context.

A new filter is added to the **Product** table **Color** column—or, the filter overwrites any filter that's already applied to the column.

Adventureworks Example

In Power BI with AdventureWorks data, USERELATIONSHIP activates an inactive relationship (e.g., Shipping Date) to calculate metrics (e.g., Shipped Amount) alongside an active one (Order Date). Use a DAX measure with CALCULATE to temporarily override the model's active relationship, allowing sales reporting based on alternate dates.

Scenario: Sales by Order Date vs. Ship Date

In the AdventureWorks model, the FactInternetSales table often links to the DimDate table via OrderDateKey (Active) and ShipDateKey (Inactive).

1. **Active Relationship:** DimDate[DateKey]

FactInternetSales[OrderDateKey]

2. **Inactive Relationship:** DimDate[DateKey]

FactInternetSales[ShipDateKey]

DAX Measure Example

To calculate the total sales amount based on the **Shipping Date** rather than the order date, create this measure:

dax

Sales Amount by Ship Date =

```
CALCULATE(  
    SUM(FactInternetSales[SalesAmount]),  
    USERELATIONSHIP(FactInternetSales[ShipDateKey], 'DimDate'[DateKey])  
)
```

Key Details

- **CALCULATE:** The USERELATIONSHIP function must be used within CALCULATE or CALCULATETABLE.
- **Override:** The measure forces the relationship, regardless of whether it is active or inactive in the model view.
- **Result:** This measure can be used in visuals to show sales by Shipping Date, while a standard SUM measure will still show Sales by Order Date.
- **Blank Values:** Note that if an order has not shipped, USERELATIONSHIP will return a BLANK value for the ship date.

Notable DAX

Using DAX more like T-SQL

PREVIOUS(), RUNNINGSUM() and MOVINGAVERAGE() functions to create comparison metrics between each fiscal year.

USERPRINCIPALNAME()

[DAX function reference - DAX | Microsoft Learn](#)

Filter Context

The **CALCULATE** function is a powerful function you can use to manipulate the filter context. The first argument takes an expression or a measure (a measure is just a named expression). Subsequent arguments allow modifying the filter context.

Dates & Fiscal Years

The **CALENDARAUTO** function returns a single-column table comprising date values. The "auto" behavior scans all data model date columns to determine the earliest and latest date values stored in the data model. It then creates one row for each date within this range, extending the range in either direction to ensure full years of data is stored.

This function can take a single optional argument that is the last month number of a year. When omitted, the value is 12, meaning that December is the last month of the year. In this case, 6 is entered, meaning that June is the last month of the year.

<https://github.com/MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst/blob/Main/Instructions/Labs/04-create-dax-calculations.md>

USERELATIONSHIP

[USERELATIONSHIP function \(DAX\) - DAX | Microsoft Learn](#)

Example: To calculate the sum of internet sales and allow slicing by ShippingDate instead of the traditional OrderDate, - **create measure** with USERELEATIONSHIP

Using [USERELATIONSHIP](#) in calculated columns requires CALCULATE to activate inactive relationships, but it is challenging due to context transition, which often triggers before the relationship change. To force it to work, you must use CALCULATE(..., USERELATIONSHIP(..), REMOVEFILTERS(..)) to ensure the correct filter context applies.

Key Considerations for Calculated Columns

- **Context Transition:** Calculated columns create a row context, and CALCULATE turns this into a filter context (context transition). This transition happens *before* USERELATIONSHIP takes effect, often resulting in incorrect data.
- **Required Formula Structure:** To properly use it, combine CALCULATE with USERELATIONSHIP and usually REMOVEFILTERS on the related table to override the default active relationship.
- **Alternative Solutions:** If USERELATIONSHIP proves too complex or fails, using {Link: LOOKUPVALUE <https://community.powerbi.com/t5/Desktop/Calculated-columns-USERELATIONSHIP-or-LOOKUPVALUE/td-p/1197385>} or DAX measures is often preferred.

Example Scenario (Date Relationship)

To calculate a delivery date year in a Sales table using an inactive relationship to a Date table:

dax

Delivery Year =

```
CALCULATE(  
    SELECTEDVALUE('Date'[Year]),  
    USERELATIONSHIP('Sales'[DeliveryDate], 'Date'[Date]),  
    REMOVEFILTERS('Date')  
)
```

REMOVEFILTERS is crucial to prevent the default "Order Date" relationship from conflicting with the "Delivery Date" activation

USERELATIONSHIP in DAX is super handy when you have **multiple relationships** between two tables but only one can be active at a time (Power BI enforces a single active relationship between any two tables). You use it inside CALCULATE to **temporarily activate an inactive relationship** for that calculation.

Below is a crisp, realistic example you can copy/paste.

Scenario

You've got a Sales fact table and a Calendar table:

- Sales[OrderDate] — when the order was placed
- Sales[ShipDate] — when the order shipped
- Calendar[Date] — your standard date dimension

Model design:

- Active relationship: Calendar[Date] → Sales[OrderDate] (Active )
- Inactive relationship: Calendar[Date] → Sales[ShipDate] (Inactive )

Why? Because most of your reports are order-based, but sometimes you need to analyze by **ship date**.

Measures

1) Base measure

Plain Text

Total Sales :=

SUM (Sales[SalesAmount])

2) Orders by Order Date (active by default)

This just works because the active relationship is on OrderDate:

Plain Text

Sales (Order Date) :=

[Total Sales]

3) Orders by Ship Date (activate the inactive relationship)

Use USERELATIONSHIP to flip to Ship Date for this calculation:

Plain Text

Sales (Ship Date) :=

CALCULATE (

[Total Sales],

USERELATIONSHIP (Calendar[Date], Sales[ShipDate])

)

Put both measures in the same visual (e.g., a line chart with Calendar[Date] on the axis) to compare performance by order vs. ship dates.

Useful Variations

A) Dynamic toggle via disconnected slicer

Let users pick "Order Date" or "Ship Date" with a slicer.

1. Create a small disconnected table Date Mode:
 - Column: DateMode with values: "Order Date", "Ship Date"
2. Measure:

DAX

Sales (Toggle Date Context) :=

VAR mode =

SELECTEDVALUE ('Date Mode'[DateMode], "Order Date")

RETURN

SWITCH (

TRUE(),

mode = "Ship Date",

CALCULATE ([Total Sales], USERELATIONSHIP (Calendar[Date], Sales[ShipDate])),

-- default: Order Date

[Total Sales]

)

``

B) Time intelligence by Ship Date

Most time-intel functions follow the active relationship. To do the same logic by Ship Date, wrap them with USERELATIONSHIP:

Last Year by Ship Date

DAX

Sales LY (Ship Date) :=

CALCULATE (

[Total Sales],

USERELATIONSHIP (Calendar[Date], Sales[ShipDate]),

```
DATEADD ( Calendar[Date], -1, YEAR )
```

```
)
```

```
..
```

Month-to-Date by Ship Date

DAX

Sales MTD (Ship Date) :=

```
CALCULATE (
```

```
[Total Sales],
```

```
USERELATIONSHIP ( Calendar[Date], Sales[ShipDate] ),
```

```
DATESMTD ( Calendar[Date] )
```

```
)
```

Key idea: keep Calendar on the axis/filter; USERELATIONSHIP injects the ship-date relationship for the calculation window.

When to use USERELATIONSHIP

- **Role-playing dimensions** (Order/Ship/Invoice dates, Contract Start/End dates, etc.)
- You need **separate measures** for each date role without creating multiple calendar tables.
- You want **explicit and readable** control of which relationship applies to a measure.

Common pitfalls & tips

- USERELATIONSHIP works only **inside** CALCULATE (or functions that invoke a calc context transition).
- It **does not** permanently change the model—only the **current measure evaluation**.
- If you need **dynamic date role switching** in visuals without multiple measures, use a **disconnected slicer + SWITCH** as shown.
- For time-intelligence patterns (e.g., YTD/LY), always apply USERELATIONSHIP in the **same CALCULATE** that wraps the time-intel function.

- If you truly need separate independent date axes (rare), consider **separate Calendar tables**—but most cases are cleaner with USERELATIONSHIP.
 - Avoid using it to “fix” a poor model. Ensure:
 - The relationships are **single-direction** from Calendar to Sales
 - Calendar[Date] is a **contiguous** date range (no gaps)
 - Your inactive relationship columns (ShipDate) have the **same data type** and appropriate granularity
-

Quick test dataset (optional)

You can paste this into Power BI as Enter Data (or load from CSV) to try:

Calendar (cover a range):

Date from 2024-01-01 to 2024-01-31

Sales

OrderID	OrderDate	ShipDate	SalesAmount
1	2024-01-03	2024-01-05	100
2	2024-01-05	2024-01-10	250
3	2024-01-10	2024-01-12	300
4	2024-01-12	2024-01-20	150

Create the active relationship on OrderDate, inactive on ShipDate, and add the measures above.

TL;DR stands for “Too Long; Didn’t Read.”

- USERELATIONSHIP(Calendar[Date], Sales[ShipDate]) inside CALCULATE lets you compute the **same measure** by an **alternate relationship** (e.g., ship date) without making it active model-wide.
- Perfect for **role-playing** date scenarios and **side-by-side** analyses (Order vs. Ship).

REMOVEFILTERS()

REMOVEFILTERS() is a **DAX table function** that clears filters from *whatever columns or tables you pass into it*.

But importantly, it **does not wipe all filters blindly** — it only removes filters on the *specified* columns/tables, and only in the *current filter context*.

Here's the full breakdown:

✓ What filters does REMOVEFILTERS() actually remove?

1. Column filters

If you pass a column, REMOVEFILTERS(Column) removes **all filter context on that specific column**, including:

- Slicer selections
- Visual-level filters
- Page-level filters
- Report-level filters
- CALCULATE modifiers
- Implicit filters from relationships (via row context transition)

Example:

DAX

Sales All Products =

```
CALCULATE([Total Sales], REMOVEFILTERS(Products[ProductName]))
```

This removes **only** ProductName filtering but keeps filtering from other columns (like Region, Date, etc.).

2. Table filters

If you pass a table, it removes filters from **every column in that table**.

DAX

Sales All Products Table =

```
CALCULATE([Total Sales], REMOVEFILTERS(Products))
```

This clears filters from:

- Products[ProductName]
- Products[Category]
- Products[Subcategory]
- Any calculated columns in the table
- Any auto-exist filters from composite selections

...but it leaves other tables' filters intact (e.g., Dates, Geography).

3. All filters through relationships (if table is removed)

By removing all filters from a table, it also removes:

- Propagated filters
- Filters via many-to-one relationships
- Filters via bi-directional relationships

Example: removing Products filters also removes **any filtering that came from related dimension tables**, because they ultimately filter Products.

! What REMOVEFILTERS() does **NOT** remove

✗ It does not remove filters from *unrelated* tables.

If Dates is not related to Products, removing Products does not clear Dates filters.

✗ It does not remove filters on measures unless those filters come from the target tables/columns.

✗ It does not change row-level security (RLS).

RLS *always* stays enforced.

✗ It does not remove filters applied by variables inside the measure, unless those variables depend on the removed column/table.

Comparison vs similar functions

Function	Removes Which Filters?	Notes
REMOVEFILTERS(Column/Table)	Only specified columns/tables	Equivalent to ALL() with cleaner semantics
ALL(Column/Table)	Same as REMOVEFILTERS()	Also returns a table; sometimes behaves differently in ambiguous models
ALLEXCEPT(Table, Columns)	Removes all filters <i>except the listed ones</i>	Opposite intent
ALLSELECTED()	Removes external filters but keeps user context	Very different behavior
KEEPFILTERS()	Adds instead of removing filters	Opposite of REMOVEFILTERS

 Important subtlety:

REMOVEFILTERS() removes both direct and propagated filters

Example:

- Visual filters Category → filters Products table
- Relationship filters Products → filters FactSales

Then:

DAX

CALCULATE([Total Sales], REMOVEFILTERS(Products))

Clears **all** of that, including the propagation into FactSales.

 Summary: What filters does REMOVEFILTERS affect?

It removes:

- Filters applied directly on the given columns/tables
- Filters coming from slicers, visuals, page/report filter panes
- Filters created by CALCULATE transition
- Filters propagated from related tables
- Auto-exist filters

It does *not* remove:

- Filters on other tables/columns you didn't specify
- RLS
- Filters on unrelated tables
- Filters created by measures unless tied to the target column/table

Toggle Slicer

PBIX kit you can use to spin up the toggle slicer scenario (**Order vs. Ship vs. Invoice**) in under 3 minutes.

Download the kit

- **ZIP (all assets):**

[tiny pbix toggle kit.zip](#)

(Contains CSVs, DAX measures, and a README with step-by-step instructions)

- If you prefer individual files:
 - [Calendar.csv](#)
 - [Sales.csv](#)
 - [DateMode.csv \(optional\)](#)
 - [DAX Measures.txt](#)
 - [README_Tiny_PBIX_Kit.md](#)

Note: I can't directly attach a .pbix from here, but this kit is designed so you can create it in Power BI Desktop with just a few clicks.

What's inside & how to use it (quick guide)

1. Load the data

- Power BI Desktop → **Get Data** → **Text/CSV** → import Calendar.csv, Sales.csv, and optionally DateMode.csv.

2. Create relationships (Model view)

- **Active:** Calendar[Date] → Sales[OrderDate] (single-direction from Calendar → Sales)
- **Inactive:**
 - Calendar[Date] → Sales[ShipDate]
 - Calendar[Date] → Sales[InvoiceDate]

3. Disconnected slicer

- If you didn't import DateMode.csv, create a calculated table:

DAX

```
Date Mode = DATATABLE(  
"DateMode", STRING,  
{ {"Order Date"}, {"Ship Date"}, {"Invoice Date"} }  
)
```

4. Add the measures

Open

[DAX Measures.txt](#) and paste them into a Measures table. The key ones:

DAX

Total Sales :=

SUM (Sales[SalesAmount])

Sales (Order Date) := [Total Sales]

Sales (Ship Date) :=

CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[ShipDate]))

Sales (Invoice Date) :=

CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[InvoiceDate]))

Sales (Toggle Date Context) :=

VAR mode = SELECTEDVALUE ('Date Mode'[DateMode], "Order Date")

RETURN

SWITCH (

mode,

"Ship Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[ShipDate])),

"Invoice Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[InvoiceDate])),

[Total Sales] // Default: Order Date

)

// Time-intel variants that respect the toggle

Sales YTD (Toggle) :=

VAR mode = SELECTEDVALUE ('Date Mode'[DateMode], "Order Date")

RETURN

```
SWITCH (
mode,
"Ship Date", CALCULATE ( [Total Sales], USERELATIONSHIP ( 'Calendar'[Date], Sales[ShipDate] ),
DATESYTD ( 'Calendar'[Date] ) ),
"Invoice Date", CALCULATE ( [Total Sales], USERELATIONSHIP ( 'Calendar'[Date], Sales[InvoiceDate] ),
DATESYTD ( 'Calendar'[Date] ) ),
CALCULATE ( [Total Sales], DATESYTD ( 'Calendar'[Date] ) )
)
```

5. Build the visuals

- Add a **Slicer** → Date Mode[DateMode] (Order/Ship/Invoice).
- Add a **Line chart**:
 - X-axis: Calendar[Date]
 - Y-axis: Sales (Toggle Date Context)
- (Optional) Add **Sales YTD (Toggle)** to compare YTD trends per mode.

Why this works

- USERELATIONSHIP **temporarily activates** the alternate date relationship (Ship or Invoice) only for the **measure** you're evaluating—your model's active relationship stays on **Order Date**.
- The **disconnected slicer** drives a SWITCH() in the measure so users can change the date role **without** duplicating visuals or creating multiple calendars.
- Time-intelligence measures (YTD/MTD/LY) piggyback on the same toggle by wrapping the appropriate USERELATIONSHIP in the **same CALCULATE** that applies the time window.

- Add more toggles (e.g., **Invoice Paid Date**, **Delivery Date**) and comparisons (e.g., **delta between Order vs. Ship**).

YTD, MTD, and Last Year (LY) versions that respect the same **Order / Ship / Invoice** toggle.

↓ YTD/MTD/LY toggles

- **ZIP (v2)**:

[tiny pbix toggle kit v2.zip](#)

Includes Calendar.csv, Sales.csv, DateMode.csv, DAX_Measures_v2.txt, and README_Tiny_PBIX_Kit_v2.md.

(If you want the original v1 as well: [tiny pbix toggle kit.zip](#))

What changed

- New measures in **DAX_Measures_v2.txt**:
 - Sales YTD (Toggle)
 - Sales MTD (Toggle)
 - Sales LY (Toggle)All three **follow the slicer** (Date Mode[DateMode]) and use USERELATIONSHIP to activate **Ship** or **Invoice** date as needed.

Copy/paste measures (for quick reference)

You can also grab them directly here:

DAX

// Year-to-Date

Sales YTD (Toggle) :=

VAR mode = SELECTEDVALUE ('Date Mode'[DateMode], "Order Date")

RETURN

SWITCH (

mode,

"Ship Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[ShipDate]),
DATESYTD ('Calendar'[Date])),

"Invoice Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[InvoiceDate]),
DATESYTD ('Calendar'[Date])),

CALCULATE ([Total Sales], DATESYTD ('Calendar'[Date]))

)

// Month-to-Date

Sales MTD (Toggle) :=

VAR mode = SELECTEDVALUE ('Date Mode'[DateMode], "Order Date")

RETURN

SWITCH (

mode,

"Ship Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[ShipDate]),
DATESMTD ('Calendar'[Date])),

"Invoice Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[InvoiceDate]),
DATESMTD ('Calendar'[Date])),

CALCULATE ([Total Sales], DATESMTD ('Calendar'[Date]))

)

// Last Year (same period)

Sales LY (Toggle) :=

VAR mode = SELECTEDVALUE ('Date Mode'[DateMode], "Order Date")

RETURN

SWITCH (

mode,

"Ship Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[ShipDate]),
DATEADD ('Calendar'[Date], -1, YEAR)),

"Invoice Date", CALCULATE ([Total Sales], USERELATIONSHIP ('Calendar'[Date], Sales[InvoiceDate]),
DATEADD ('Calendar'[Date], -1, YEAR)),

CALCULATE ([Total Sales], DATEADD ('Calendar'[Date], -1, YEAR))

)

Quick steps (recap)

1. **Load** Calendar.csv, Sales.csv, DateMode.csv.
2. **Relationships:** Active = Calendar[Date] → Sales[OrderDate]; Inactive = Calendar[Date] → Sales[ShipDate] & Sales[InvoiceDate].
3. **Paste** the measures from

[DAX Measures v2.txt](#).

4. Drop a **Slicer** for Date Mode[DateMode] and use Sales (Toggle Date Context) or the new YTD/MTD/LY measures in visuals.

Parameters

1. [Use Parameters to Visualize Variables - Power BI | Microsoft Learn](#)
 - Create a parameter
 - Create a measure by using a formula that includes the parameter.
 - Create a visual that changes as you use a slicer to adjust the parameter value.
2. [Filter a report using query string parameters in the URL - Power BI | Microsoft Learn](#)
 - When you open a report in Power BI service, each page of the report has its own unique URL.
 - To filter that report page, you could use the Filters pane on the report canvas.
 - Or you could add query string parameters to the URL to prefilter the report.

Quick chooser: which “parameter” should I use?

- **Switch server/database, file path, or API URL?** → *Power Query parameter*
- **Filter at the source with a slicer?** → *Dynamic M query parameter*
- **Let users switch measures/axes?** → *Field parameter*
- **Numeric scenario analysis (slider)?** → *What-if parameter*
- **Partition/refresh control?** → *Incremental refresh parameters*
- **Pixel-perfect reports with prompt inputs?** → *Paginated report parameters*

Power BI parameters can be used in several ways beyond the basic “what-if” pattern (create parameter → use in a measure → slicer controls value → visual updates). Below is a concise map of the parameter types and the *other* places you can use them, with practical examples and gotchas.

1) Power Query Parameters (M) — for data shaping, environments, and governance

Where they work

- **Data source switching** (Dev/Test/Prod connection strings, database names, schema names)
- **Credentials and paths** (server name, SQL database, file paths, API base URLs)
- **Reusable constants in M logic** (e.g., filters, limits, column selections)
- **Dataflows** (same support in the Service; can be edited after publish)
- **Deployment pipelines** (map parameter values per stage via rules)
- **Incremental refresh** (tie to RangeStart/RangeEnd datetime parameters)

Examples

- **Environment switcher**

Power Query

```
// Parameter: Environment = "Dev" | "Prod"
```

```
// Parameter: DevServer = "dev-sql"
// Parameter: ProdServer = "prod-sql"
Source = Sql.Database(
if Environment = "Prod" then ProdServer else DevServer,
"SalesDW"
)
```

- **API base URL**

Power Query

```
// Parameter: ApiBaseUrl = "https://api.contoso.com/v1"
Source = Json.Document(Web.Contents(ApiBaseUrl & "/orders?top=1000"))
```

Notes

- These parameters are **editable in the Service** (Dataset settings → Parameters) without republishing.
- They're not for DAX; they live in M (Power Query) and affect data retrieval/transformations.

2) Dynamic M Query Parameters — push filters down to the source

Bind a slicer to a field and have it **drive a Power Query parameter at query time** (for supported DirectQuery sources like SQL Server, Snowflake, Synapse, etc.). This lets the filter be **folded** to the source, reducing data movement.

Use cases

- Switching between **large fact partitions** (e.g., Region = US/EU)
- **Server-side search** / Top N (avoid importing entire dataset)
- **Role-based** or **tenant-aware** queries (with caution)

Example concept

- Create a PQ parameter SelectedCountry.
- Enable **Dynamic M Query Parameters** and bind SelectedCountry to a model column.
- Use it in your native SQL:

Power Query

```
Value.NativeQuery(
Sql.Database(Server, Db),
```

```
"SELECT * FROM Sales WHERE Country = @country",  
[Parameters = [country = SelectedCountry]]  
)
```

Notes

- Works best with DirectQuery and foldable sources.
- Unlike What-if, the slicer drives data retrieval, not just visuals.

3) Field Parameters — switch fields/measures dynamically (no DAX calc groups needed)

What they are: A model table that lets users switch **dimensions/measures** via slicer. Super useful for building one visual that can pivot among multiple KPIs or categories.

Use cases

- **Dynamic axis or legend** (e.g., Month vs. Quarter vs. Year)
- **Dynamic measure selection** (e.g., Revenue vs. Margin vs. Units)
- **Dynamic detail drill** (compact UX instead of multiple report pages)

Example

- Create a Field Parameter with [Revenue], [Gross Margin %], [Units].
- Put it in a slicer and into your visual's **Y-axis**. The visual switches measure as the slicer changes.

Notes

- Field parameters can be used across visuals, conditional titles, tooltips, etc.
- They complement what-if parameters; they don't store scalar values—they **switch fields**.

4) What-If Parameters — beyond a single measure

You already know the basic pattern, but they can do more:

- **Drive conditional formatting** (colors, data bars, thresholds)
- **Dynamic titles/annotations** (refer to parameter value)
- **Scenario selections** (pair with SWITCH logic across multiple measures)
- **Sensitivity analysis** (multiple what-ifs combined)

Example

Power Query

Price Sensitivity = GENERATESERIES(0.8, 1.2, 0.05) // creates a table

Selected Factor = SELECTEDVALUE('Price Sensitivity'[Price Sensitivity], 1)

Adj Revenue = [Base Revenue] * [Selected Factor]

5) Incremental Refresh Parameters — governed refresh windows

- Special **datetime parameters** RangeStart and RangeEnd filter your fact table in Power Query.
- Power BI uses them to create partitions and refresh only recent data.

Example M filter

Power Query

FilteredRows = Table.SelectRows(Source, each [OrderDate] >= RangeStart and [OrderDate] < RangeEnd)

Notes

- These are parameters, but with a specific meaning for partitioning/refresh.
-

6) Paginated Reports (Report Builder) — classic parameters & integration

- **True report parameters:** prompt users, cascade parameters, pass between reports.
 - From Power BI, you can **drill-through** to a paginated report and **pass filters as parameters**.
 - Excellent for **pixel-perfect** exports and **large-detail** tables.
-

7) URL & Button-driven “parameters” for navigation

While not Power Query parameters:

- **URL filters** (querystring) can pre-filter a Power BI report.
 - **Bookmarks + Buttons** can simulate **scenario selection** or parameter presets.
 - Use with **Field Parameters** to create “view modes” (e.g., Summary vs. Detail).
-

8) Dataflows & Fabric (if applicable)

- **Dataflows Gen1/Gen2** support PQ parameters for reusable transformations.
 - Parameters help centralize **source/tenant/env** variables across many datasets.
-

9) Governance & DevOps

- **Deployment pipelines:** define rules to **map parameter values per stage** (Dev → Test → Prod).
 - **Service APIs / XMLA:** admins can read/update parameter values at scale (automation).
-

Common Pitfalls & Best Practices

- **Name carefully** (e.g., Env_Server, Api_BaseUrl, RangeStart/RangeEnd); keep a parameter dictionary in your project README.
- **Don't overuse what-if tables**; too many can bloat the model. Prefer **field parameters** for switching semantics.
- **Use dynamic M query parameters** for **large DirectQuery models** to reduce data movement.
- **Secure by design**: PQ parameters are not a substitute for **RLS**. Don't rely on them to hide data—enforce RLS at the model/source.
- **Service edits**: Remember that **Power Query parameters** are editable post-publish—great for ops, but coordinate changes.

Use Power Query Parameters for Server/DB/File/API Switching

Power Query parameters live in the **M** layer and allow you to dynamically change your **data source**, **file paths**, and **API endpoints** without touching the DAX model.

Here's how to do it cleanly.

1) Create the parameter

Example: environment name parameter

1. Home → **Manage Parameters** → **New Parameter**
2. Name: Environment
3. Type: Text
4. Allowed values: List
 - Dev
 - Test
 - Prod
5. Current value: Dev

Optional helper parameters (recommended)

You can also create parameters for each environment's server:

- DevServer = "dev-sql.company.com"
- TestServer = "test-sql.company.com"
- ProdServer = "prod-sql.company.com"

Same idea for:

- Database names
- File paths
- API base URLs

2) Use the parameter inside your M code

Example: switching SQL Server based on environment

Power Query

```
// Choose server based on the Environment parameter
```

```
ServerName =
```

```
if Environment = "Prod" then ProdServer
```

```
else if Environment = "Test" then TestServer
```

```
else DevServer,
```

```
Source = Sql.Database(ServerName, "SalesDW")
```

```
``
```

Example: switching file paths

Power Query

```
Path =
```

```
if Environment = "Prod" then "C:\Prod\Files\"
```

```
else if Environment = "Test" then "C:\Test\Files\"
```

```
else "C:\Dev\Files\",
```

```
Source = Csv.Document(File.Contents(Path & "Sales.csv"))
```

Example: switching API base URL

Power Query

```
BaseUrl =
```

```
if Environment = "Prod" then ProdApi
```

```
else DevApi,
```

```
Source = Json.Document(  
Web.Contents(BaseUrl & "/orders?top=100")  
)
```

This lets you move the entire solution across environments without editing queries.

3) Publish and edit parameters in the Power BI Service

After publishing:

In the Service:

Dataset → **Settings** → **Parameters**

From here you can choose:

- Dev
- Test
- Prod

...then refresh the dataset.

This allows **environment switching without republishing** the PBIX file.

4) (Optional) Use deployment pipelines for automated mapping

In a Power BI Deployment Pipeline:

- Go to **Parameter Rules**
- Map:
 - Dev stage → Dev parameters
 - Test stage → Test parameters
 - Prod stage → Prod parameters

This gives you **automatic environment switching** during deployment.

5) Best Practices

✓ **Use descriptive parameter naming**

Environment, ProdServer, DevApiBaseUrl, etc.

✓ Do NOT hardcode the server/path in queries

Always reference the parameter.

✓ Use parameters for everything that might change

- SQL server
- Database
- API URL
- File folder
- Filename
- Authentication method

✓ Document all parameter values in a README or wiki**✓ Avoid using PQ parameters for row-level security**

They aren't secure for hiding user-specific data.

Power BI Forecasting

a **clean, practical breakdown** of the **actual math** behind Power BI's Forecasting (line charts) so you know exactly how it calculates **units + seasonality** for **weekly, monthly, and multi-year** forecasts.

This is **not** Microsoft's proprietary code, but the math is public and documented because Power BI uses **ETS (Exponential Triple Smoothing)**, also called **Holt-Winters** — specifically the **ETS(A, A, A)** additive model.

✓ 1. Power BI Forecasting = ETS (Holt-Winters Triple Exponential Smoothing)

Power BI uses the statistical algorithm:

Forecast = Level + Trend + Seasonality

Mathematically:

1. Level (L_t)

$$L_t = \alpha(Y_t - S_{t-m}) + (1 - \alpha)(L_{t-1} + T_{t-1})$$

2. Trend (T_t)

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

3. Seasonality (S_t)

$$S_t = \gamma(Y_t - L_t) + (1 - \gamma)S_{t-m}$$

4. Forecast h periods ahead

$$\hat{Y}_{t+h} = L_t + hT_t + S_{t-m+(h \bmod m)}$$

Where:

- Y_t = your historical units
- m = season length (Power BI infers this when you choose Seasonality = Auto)
- α, β, γ = smoothing coefficients (Power BI automatically optimizes them)

✓ 2. What “Units + Seasonality Math” Means in Practice

Let's make this less abstract.

◆ When Power BI sees weekly patterns

For **weekly** data:

- **m = 7**
(Power BI detects 7-day cycles)

Seasonality term rotates every 7 data points.

◆ **When Power BI sees monthly patterns**

For **monthly** data:

- **m = 12**
(12 months in a seasonal cycle)

◆ **When your dataset is years long**

If you forecast **years**, the model still uses:

- **m = 12** (monthly)
- or **m = 52** (weekly)
- or **m = 365** (daily)

depending on the underlying grain.

Power BI automatically picks this based on your axis granularity.

✅ **3. The key math differences: Week vs. Month vs. Year**

The formulas don't change —
only the value of m (season length) changes.

 If forecasting weekly

m = 7

Meaning: pattern repeats every 7 points.

Example: sales spike every weekend.

Forecast logic:

- Level: current baseline
 - Trend: slope over the last several weeks
 - Seasonality: 7-point repeating pattern
-

 If forecasting monthly

$m = 12$

Example: seasonality repeats each year (Jan, Feb, Mar...).

Forecast logic:

- Level: baseline units
 - Trend: long-term direction
 - Seasonality: 12 repeating monthly adjustments
-

 If forecasting across multiple years

Power BI still uses **single season length** based on the lowest axis grain.

If the X-axis uses **years as display**, but your data is monthly:

- Power BI uses **monthly seasonality (12)**
- Then aggregates the results into the chart

So the forecast is *still* based on monthly patterns.

 4. How Power BI decides “seasonality” automatically

Power BI uses an autocorrelation-driven selection:

1. It computes candidate seasonalities using:
 - ACF (autocorrelation function)
 - MAPE reduction heuristics
2. Tests multiple m values
3. Picks the one giving best likelihood & lowest error

You can override this manually.

 5. Practical example

Say you have **units sold** daily:

Y_t = units each day

$m = 7$ (weekly seasonality)

Power BI computes:

- $L_t \rightarrow$ baseline daily units
- $T_t \rightarrow$ recent trend (increasing or decreasing)
- $S_t \rightarrow$ repeating weekly cycle (Mon–Sun adjustment)

Forecast for 14 days:

$$\hat{Y}_{t+h} = L_t + hT_t + S_{(h \bmod 7)}$$

If sales spike on Friday, the forecasted Fridays will also spike.

✓ 6. TL;DR — What math drives Power BI forecasting?

Concept	Meaning
Model	ETS Triple Exponential Smoothing (Holt-Winters Additive)
Seasonality (m)	Based on axis granularity: 7=weekly, 12=monthly, 365=daily
Forecast	= Level + Trend + Seasonal component
What you control	season length, confidence interval, forecast horizon
What Power BI auto-optimizes	α , β , γ (the smoothing multipliers)

Shared Semantic Model Datasets

Depending on your data and report, you can do an adhoc data refresh anytime or set a schedule. However, scheduled refreshes require gateways.

“Shared dataset” behavior is essentially a Power BI Service/Fabric feature.

But there are a few **workable offline/desktop-only patterns** depending on what you need.

What you can do without the Service

1) Model-only PBIX (no report pages)

- You can absolutely build a **dataset/model-only** PBIX in Desktop (just delete report pages).
- You can **share the PBIX file** itself with someone else. When they open it in their own Desktop, they can create visuals **inside that file**.
- **Limitation:** This is *not* a “shared dataset.” Each person is editing their own **copy** (no central model; no thin reports that point to a single dataset).

Good for: handoff of a reusable model as a file, one author at a time.

2) Connect one Desktop to another Desktop's open model (same machine)

- In Power BI Desktop, **Get data → Power BI datasets** shows a **“This PC”** section for models currently open in Desktop on **that machine**.
- You can open a second Desktop instance and create a **thin report** that live-connects to the model in the first instance.
- **Limitations:**
 - This works only **on the same PC** (not a network-wide server).
 - The first Desktop instance must remain **open**; it's not a durable, shareable endpoint.
 - Not practical for team collaboration across machines.

Good for: a local “thin report” dev pattern or quick self-checks.

3) SSAS Tabular (on-prem) as the shared semantic model

- If you want a **true shared dataset experience off the cloud**, use **SQL Server Analysis Services (SSAS) Tabular** on a server you control.
- Power BI Desktop can **live connect** to SSAS Tabular just like it does to a Power BI Service dataset.
- Multiple report authors can build thin reports against the same central model.

- **Tradeoffs:** Requires SSAS setup/ops, role/security management, and deployment pipeline for the Tabular model (e.g., via Visual Studio/Tabular Editor).

Good for: on-prem “shared dataset” equivalent without Power BI Service.

<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-analysis-services-tabular-data>

What is unique to the Service/Fabric

- **Shared datasets** that multiple thin reports connect to
- **Centralized discovery** (lineage, endorsement, certified datasets)
- **Row-Level Security** enforcement at scale with auditability
- **XMLA endpoint** for enterprise modeling (Premium/Fabric)
- **Deployment pipelines**, governance, and workspace permissions
- **Scheduled refresh** and managed gateways (for on-prem sources)

These are the cloud-native capabilities designed for **team-wide reuse** and lifecycle management.

Summary Matrix

Goal	Works with just files?	How to do it	Caveats
Reuse a dataset by handing off a file		Share a model-only PBIX	Not a shared dataset; recipients edit their own copy
Build a thin report against another Desktop model	 Local only	Open PBIX #1 (model), then Connect to “This PC” from PBIX #2	Same PC only; the first Desktop must stay open
True “shared dataset” off cloud	 (server needed)	SSAS Tabular ; connect Power BI Desktop live	Requires SSAS infrastructure & deployment process
True “shared dataset” in cloud	 Best practice	Publish dataset to Power BI Service/Fabric ; build thin reports	Requires Service; brings governance features

Practical recommendations

- If you just need to **hand someone a model** to build reports:
→ Save a **model-only PBIX** (or a **PBIT template**) and share the file.
- If you need multiple authors to **reliably reuse one central model without** the Service:
→ Stand up **SSAS Tabular** and have everyone live-connect to it.
- If you're okay using the cloud and want governance, RLS, pipelines, discovery, etc.:
→ Publish the dataset to **Power BI Service/Fabric** and build thin reports against it (the canonical pattern)

Dashboards/Workspaces

- Pinned visuals are set with filter context at time of pin.
- If the underlying visual changes, you'll need to update the dashboard tile as well.
- For time-based filters, it's a better idea to use a relative date slicer (or, Q&A using a relative time-based question).

	Dashboard	Workspace
Purpose	View & monitor KPIs	Create, manage, and store PBI content
What it contains	Pinned visuals from reports	Reports, dashboards, datasets, dataflows
Who can access	End users (view only)	Creators, contributors, admins
How users interact	Consume, share, set alerts	Build & edit content, schedule refresh
Update cadence	Updates when underlying report updates	Updates when content is modified or refreshed

RLS Row Level Security – Report vs Fabric/Service

Row-level security (RLS) with Power BI can be used to restrict data access for given users.

*Perform post-publication task to map security principals to the **Salespeople** role.*

- Filters restrict data access at the row level, and you can define filters within roles. In the Power BI service, users with access to a workspace have access to semantic models in that workspace.
- RLS only restricts data access for users with Viewer permissions. It doesn't apply to Admins, Members, or Contributors.
- You can configure RLS for data models imported into Power BI with Power BI.
- You can also configure RLS on semantic models that are using DirectQuery, such as SQL Server.
- For Analysis Services or Azure Analysis Services live connections, you configure row-level security in the model, not in Power BI.

The security option doesn't show up for live connection semantic models.

1. [Securing Power BI with Advanced RLS & OLS | Rysun](#)
2. [Row-level security \(RLS\) with Power BI - Microsoft Fabric | Microsoft Learn](#) - Service
3. [Manage and secure Power BI - Training | Microsoft Learn](#) – Report
4. [Row-level security \(RLS\) with Power BI - Microsoft Fabric | Microsoft Learn](#) – Model
5. Static vs Dynamic Options: userprincipalname() function will compare the email address from the Employees table with the email that the user entered when signing in to Power BI service.
6. SSO: Azure SQL Database for their sales operations that resides in the same tenant as Power BI. The database enforces RLS to control access to rows in various database tables. You can create a DirectQuery model that connects to this database without roles and publish it to the Power BI service. When you set the data source credentials in the Power BI service, you enable SSO. When report consumers open Power BI reports, Power BI passes their identity to the data source. The data source then enforces RLS based on the identity of the report consumer.
7. Object-level security (OLS) can restrict access to specific tables and columns, and their metadata.
8. The table includes columns that store employee name, phone, email address, and salary. While general report consumers can see employee name and contact details, they must not be able to see salary values. Only senior Human Resources staff are permitted to see salary values. So, the data modeler used OLS to grant access to the salary column only to specific Human Resources staff.

Object Level Security – RLS Advanced

1. [Get started with Tabular Model Definition Language \(TMDL\) | Microsoft Learn](#)
2. To turn on this public preview feature, go to File > Options and settings > Options > Preview features and check the box next to TMDL View.
3. When a user opens a Power BI report that queries a restricted object (for them), the error message could be confusing and will result in a negative experience. To them, it looks like the report is broken. A better approach might be to create a separate set of models or reports for the different report consumer requirements.

Azure Functions

1. Functions in multiple languages such as **C#, Java, JavaScript, Python, PowerShell, and Java**. Functions are triggered by events like HTTP requests, database changes, message queues, timers, or file uploads, and can connect to other Azure services through **bindings** without extra boilerplate code.
2. [Azure Functions overview | Microsoft Learn](#)
3. [Azure Functions Scenarios | Microsoft Learn](#)

Azure Dataverse

4. [Add a Microsoft Dataverse database - Power Platform | Microsoft Learn](#)
5. [Licensing overview for Microsoft Power Platform - Power Platform | Microsoft Learn](#)
6. [Power Platform Licensing Guide](#) – pdf
7. [Connect to Dataverse with model context protocol \(MCP\) - Power Apps | Microsoft Learn](#)
 - Model Context Protocol (MCP) is an open protocol that enables seamless integration between large language model (LLM) applications and external data sources and tools.
8. [Billing rates and management - Microsoft Copilot Studio | Microsoft Learn](#)

Centralized Data Ecosystem

case-study style comparison for how a **government entity** (GCC / GCC High / DoD) with **Azure, Microsoft Fabric, and Microsoft 365/O365** could build a **central data store** to power enterprise-grade **Power BI semantic models**.

The answer uses evidence from Microsoft-published and industry sources to ensure accuracy.

Case Study: Building a Central Government Data Store for Power BI Using Azure, Fabric & O365

A U.S. government agency wants to consolidate data scattered across line-of-business apps, legacy SQL systems, SharePoint lists, and spreadsheets. They already own:

- Azure Government subscription
- Microsoft Fabric capacity
- Microsoft 365 GCC / GCC-High
- Power Platform (Dataverse available)

Their goal: **Create a secure, governed, scalable central data store to service Power BI semantic models and reports.**

The agency is evaluating **three main options**:

1. **Microsoft Dataverse** (Power Platform SaaS data service)
2. **Azure SQL Database** (PaaS relational engine)
3. **Lake-centric architecture via Microsoft Fabric OneLake** (SaaS-based unified analytics platform)

1. Option A — Microsoft Dataverse

Dataverse is **not just a database**—it is a full **application data platform**: security model, API layer, business rules, and ALM included “out-of-the-box”. [\[community....atform.com\]](#)

Why a government entity might choose Dataverse

- Built-in **RBAC**, auditing, encryption at rest & in transit, and compliance certifications (including **FedRAMP High**, DoD IL4/IL2 for Power Apps US Gov). [\[learn.microsoft.com\]](#)
- Strong integration with **Power Apps**, **Power Automate**, and **M365** (Teams, SharePoint integration, model-driven apps). [\[community....atform.com\]](#)
- Ideal for transactional workloads with business processes.

Limitations (particularly for large analytic workloads)

- Storage is more expensive than raw Azure services or lake storage. Additional storage is billed when quotas exceed allocations (tenant-pooled). [\[airbyte.com\]](#)
- Not designed for **high-volume analytics** like IoT, logs, or multi-TB fact tables.
- Performance/scale constraints vs. Azure SQL for deep analytical querying (Microsoft’s own internal evaluation emphasizes this tradeoff). [\[microsoft.com\]](#)

Costs / Billing Units

- **Dataverse Developer Plan** — free (but not for production). [\[airbyte.com\]](#)
- **Dataverse Premium** — **\$20/user/month**, includes 250 MB database + 2GB file storage per user, pooled at tenant level. [\[airbyte.com\]](#)
- Additional storage incurs **extra charges** when exceeding entitlements. [\[community....atform.com\]](#)

Is it free at rest?

✗ No — Dataverse storage beyond the included entitlements is a **paid resource**.

2. Option B — Azure SQL Database (PaaS)

Azure SQL provides a **scalable relational engine** suitable for structured data, complex joins, and enterprise BI modeling. Government entities can run SQL in **Azure Government** with FedRAMP authorization (via LSP agreements). [\[azure.microsoft.com\]](#)

Why a government entity chooses Azure SQL

- Mature relational store for **enterprise data modeling** and serving semantic models.
- Multiple deployment models: **Single DB**, **Elastic Pools**, **Managed Instance** (supports more SQL Server features).

- Full VNet isolation, firewall restrictions, and network security capabilities for regulated workloads (e.g., CJI, FedRAMP). [\[it2edu.com\]](https://it2edu.com)
- Predictable, high-performance analytical querying.

Limitations

- Requires DBA/data engineering skills.
- No low-code features (business rules, forms, workflows) like Dataverse.
- Governance/lineage must be built or handled in Fabric/Purview.

Costs / Billing Units

Billing is based on compute + storage:

- vCore model: compute + storage priced separately; **serverless** supports per-second billing for intermittent workloads. Azure SQL Database pricing is fully transparent and based on vCores, IO, and storage capacity. [\[azure.microsoft.com\]](https://azure.microsoft.com)
- DTU model: bundles compute and IO; monthly cost depends on tier (Basic, Standard S1/S3, Premium P1/P4) with example costs ranging from **\$4.90 to over \$1,860 per DB per month** depending on tier and storage allocated. [\[azurelessons.com\]](https://azurelessons.com)

Is it free at rest?

✗ No — Azure SQL is **metered** for compute + storage (GB/month).

3. Option C — Microsoft Fabric OneLake (SaaS Lakehouse for Analytics)

Fabric unifies **data lake, warehouse, engineering, real-time analytics, and Power BI** into a single SaaS platform with built-in governance, metadata, security, and OneLake storage. [\[learn.microsoft.com\]](https://learn.microsoft.com)

Why a government entity chooses Fabric / OneLake

- Centralized “**single lake**” architecture enabling multi-department data sharing through OneLake federated storage and shortcuts (lakehouse + warehouse). [\[msfabric.pl\]](https://msfabric.pl)
- Strong native governance built into the platform, plus optional Purview integration for agency-wide governance and auditing. [\[learn.microsoft.com\]](https://learn.microsoft.com)
- Eliminates ETL duplication between lake and warehouse—great for Power BI semantic models.
- Works well for **multi-TB** datasets, logs, IoT, real-time feeds.

Limitations

- Not a transactional database; not a replacement for OLTP (Dataverse/SQL still needed for app write workloads).
- Requires Fabric capacity (SKU) — cost is fixed monthly, not usage-based.

Costs / Billing Units

- Governed by **Fabric Capacity Units (CU)**.
- OneLake storage is charged as **Azure Data Lake Storage Gen2** (standard storage rates apply) as Fabric rests on ADLS Gen2 per documentation. [\[msfabric.pl\]](https://msfabric.pl)
- Fabric governance features included; additional Microsoft Purview may require an extra license depending on features used. [\[learn.microsoft.com\]](https://learn.microsoft.com)

Is it free at rest?

✗ No — OneLake (Fabric) uses **metered Azure Storage costs** (hot/cool tiers depending on configuration), and Fabric capacity is billed.

Comparison Summary for a Government Central Data Store

High-Level Table

Requirement	Dataverse	Azure SQL DB	Microsoft Fabric (OneLake + Warehouse)
Best for	Apps, transactions, workflow, M365-integrated business data	Relational modeling, master data, BI semantic store	Lakehouse, analytics at scale, multi-modal data
Max Scale	Moderate (GBs)	High (TBs)	Very High (multi-TB to PB)
Compliance	FedRAMP High, IL4/IL2 (Power Apps Gov) [learn.microsoft.com]	Azure Gov FedRAMP	Fabric uses Azure security + Purview integration; supports gov compliance needs [p3adaptive.com]
Governance	Strong RBAC + built-in logic	Requires Purview / Fabric	Strong built-in governance + Purview option
Power BI Integration	Strong for business apps; moderate for enterprise BI	Excellent for semantic models	Native (Fabric Warehouse/Semantic models)
Cost Model	Per-user + storage	Compute + storage	Capacity units + storage
Free at Rest?	✗ No	✗ No	✗ No

Recommended Architecture for the Government Agency

Tiered Model (Best Practice)

1. Operational Layer (Write-heavy)

Use **Dataverse** or **Azure SQL** (depending on the system):

- Dataverse for Power Apps-based case management, permitting, inspections, HR workflows.
- Azure SQL for high-volume transactional systems, legacy migrations, master data hubs.

2. Analytical Lakehouse Layer

Use **Microsoft Fabric OneLake** for:

- Raw → cleaned → curated layers
- Data warehouse workloads
- Real-time analytics
- All data transformations
- Power BI semantic models (native in Fabric)

Fabric's design enables government agencies to **unify all analytics workloads**, modernize legacy data flows, and meet compliance frameworks for public sector environments. [\[p3adaptive.com\]](https://p3adaptive.com)

3. Governance Layer

- Use **Fabric governance + Purview** for metadata lineage, DLP, classification, policy enforcement across the entire estate.
- Manage Fabric through the admin portal for tenant, domain, and workspace governance rules. [\[learn.microsoft.com\]](https://learn.microsoft.com)

Final Recommendation

For a U.S. government entity with Azure + Fabric + O365:

Use a **hybrid approach**:

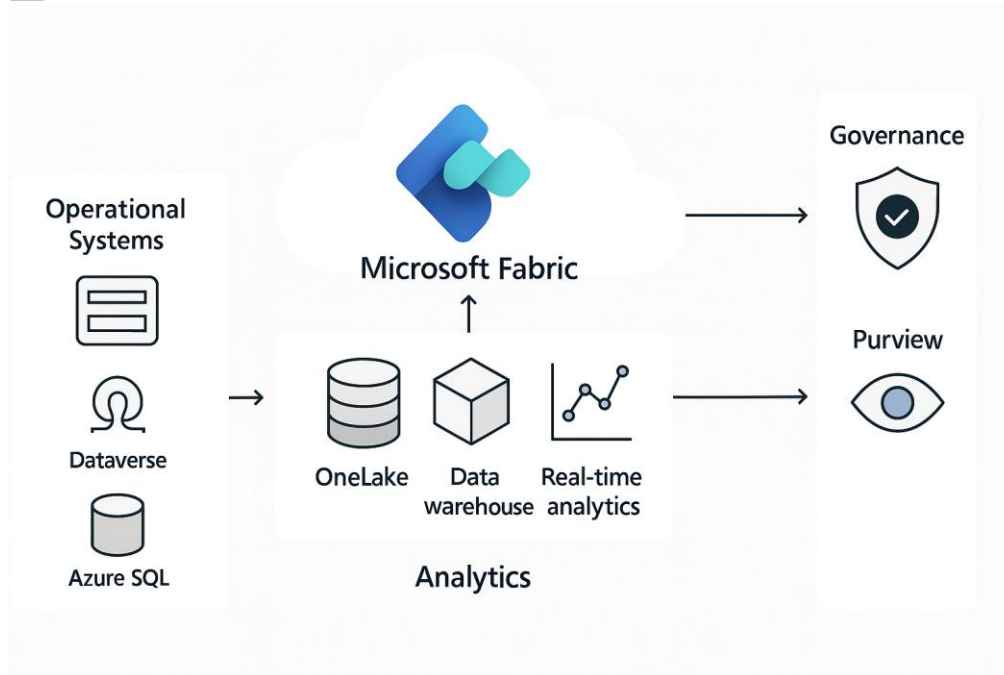
- **Dataverse** for low-code business processes, forms, auditing, secure RBAC, and workflow apps.
- **Azure SQL** for highly relational enterprise data and transactional workloads feeding the analytics layer.
- **Microsoft Fabric / OneLake** as the **central analytics store** powering **Power BI semantic models**, dashboards, real-time analytics, and data science needs.

This gives the agency:

- Full compliance support
- Centralized, governed analytics

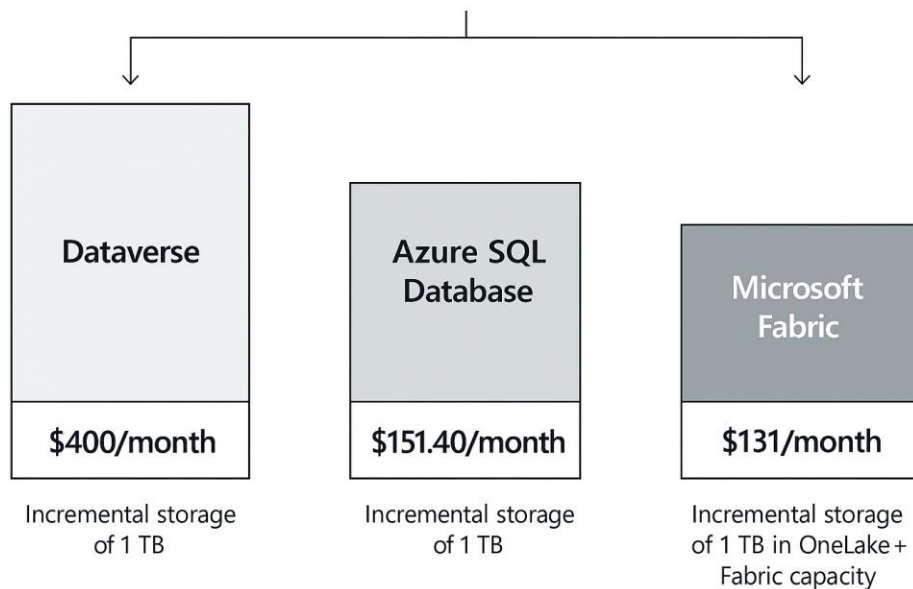
- Modernized lakehouse architecture
- Independence between transactional and analytics workloads
- Optimized costs

✓ An architecture diagram (Fabric + SQL + Dataverse integration)



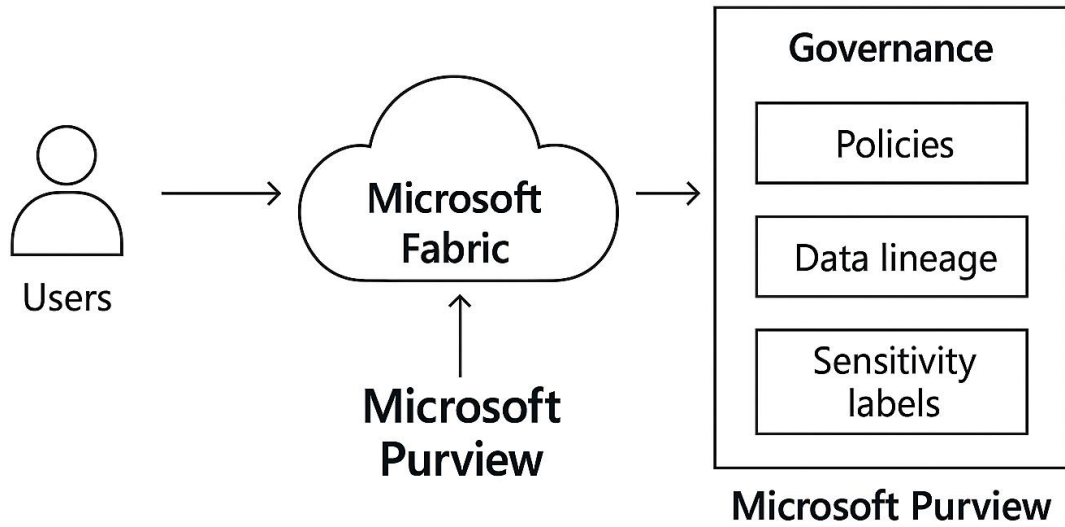
✓ A cost-estimation model comparing all three options

Estimated Costs



- ✓ A governance blueprint using Purview + Fabric

[Learn about data governance with Microsoft Purview | Microsoft Learn](#)



Reference

[Exam PL-300: Microsoft Power BI Data Analyst - Certifications | Microsoft Learn](#)

9. [Course PL-300T00--A: Microsoft Power BI Data Analyst - Training | Microsoft Learn](#)
10. [GitHub - MicrosoftLearning/PL-300-Microsoft-Power-BI-Data-Analyst](#)
11. [UseRelationship or Role-Playing Dimension; Dealing with Inactive Relationships in Power BI - RADACAD](#)
12. [Relationship functions \(DAX\) - DAX | Microsoft Learn](#)
13. [USERELATIONSHIP function \(DAX\) - DAX | Microsoft Learn](#)
14. [USERELATIONSHIP and direction of context propagation | Javier Guillén](#)
15. [Use Parameters to Visualize Variables - Power BI | Microsoft Learn](#)
16. [dotnet new <TEMPLATE> - .NET CLI | Microsoft Learn](#)

More Filtering

17. [USERELATIONSHIP and direction of context propagation | Javier Guillén](#)

Templates

Azure Resource Manager templates are JavaScript Object Notation (JSON) files that define the infrastructure and configuration for your project.

18. [AI app templates | Build AI applications with pre-made templates](#)
19. [ARM template documentation | Microsoft Learn](#)

Analyzing & Large Report Example

1. [**Data, Probability and Statistics \(mathsisfun.com\)](#)
2. [Ship INSIGHT \(Container Ship\) Registered in Marshall Is - Vessel details, Current position and Voyage information - IMO 9790907, MMSI 538007806, Call Sign V7ZZ9 | AIS Marine Traffic](#)
3. [**Data, Probability and Statistics \(mathsisfun.com\)](#)
4. endlist